

CANNBench x pyasc-skill-stack

Что было передано evaluator'у, какие операторы реально оценивались, как создавались реализации и что CANNBench может — и не может — подтвердить о происхождении кода.

9

операторов в submission

178/180

пройдено открытых
case'ов

592.87

суммарный score выборки

0

anti-cheat failures

Содержание

1. Резюме и прямые ответы	3
2. Позиционирование CANNBench	4
3. Что было «скормлено» evaluator’у	5
4. Охват и список операторов	6
5. Что именно измеряет CANNBench	8
6. Как создавались ядра	9
7. Атрибуция и anti-cheat	11
8. Результаты аппаратного прогона	13
9. Ограничения и интерпретация	16
10. Рекомендации для следующего submission	17
Приложения: evidence, хеши и источники	18

Статус документа. Это отчёт о реально выполненном private run, а не заявка на место в публичном leaderboard. Все численные результаты взяты из сохранённого финального JSON CANNBench; происхождение кода восстановлено по локальным prompt, worker-log, candidate и digest артефактам.

1. Резюме и прямые ответы

Evaluator получил не модель и не skill-stack, а самодостаточный ZIP-проект, который собирал Python wheel `cann_bench` с девятью операторными интерфейсами и vendored runtime `pyasc v2`.

Что оценивалось

Восемь операторов L1 — полный опубликованный L1-набор — и один оператор L2, RMSNorm. Всего 180 открытых case'ов: по 20 на оператор.

Что CANNBench видел

Исходники Python-модулей, build scripts, vendored `pyasc/pybind11` и итоговые запускаемые NPU kernels. Prompt'ы, worker sessions и логи генерации в ZIP не входили.

Что CANNBench измерял

Собираемость/запускаемость, соответствие Golden и время device-kernel на реальном Ascend 950PR относительно baseline и НАР-якоря.

Знает ли он, что код сгенерирован

Нет, не из кода. Атрибуция модели и Agent/Skill — декларативные metadata. В этом run поля `base_model` и `agent_skill` остались пустыми.

Главный вывод по атрибуции. Ноль anti-cheat нарушений означает, что evaluator не обнаружил запрещённый способ вычисления. Это не является криптографическим доказательством, что исходники создала конкретная модель. Для доказуемого сравнения «GLM-5.2 + pyasc-skill-stack» нужен отдельный provenance manifest и заполненные metadata.

Итог аппаратного прогона

- **178/180** case'ов прошли; pass rate 98.89%.
- **7 из 8 L1** и RMSNorm прошли 20/20; SwiGlu — 18/20.
- **overall score 592.8710**; geometric-mean speedup 0.5772x.
- **2 оператора** в среднем быстрее опубликованного baseline: MaskedScale и ForeachAddcddivScalar.
- Job получил terminal status `correctness_failed`, поскольку CANNBench требует прохождения correctness gate для всей выбранной выборки.

2. Позиционирование CANNBench

CANNBench позиционируется как открытый, versioned benchmark для AI-generated operator kernels на Huawei Ascend NPU. Его задача — дать общую аппаратно привязанную шкалу для сравнения базовых моделей, агентов и Agent/Skill-подходов независимо от конкретного рецепта обучения.

Официальная формулировка в практическом смысле: единая спецификация задачи → проверка точности → измерение kernel time на реальном Ascend NPU → сравнение по score, speedup и coverage. Это benchmark способности системы создавать операторную реализацию, а не benchmark скорости inference самой LLM.

Vendor-aligned

Спецификации, Golden, cases и baselines поддерживаются рядом с официальным CANN repository.

Agent-oriented

Score задуман как reward signal для итеративного цикла генерации, проверки и оптимизации kernels.

Hardware-grounded

Performance измеряется на NPU и привязывается к baseline и аналитическому HAP limit.

Ссылки на позиционирование

1. [Официальный портал CANNBench](#) — краткое позиционирование, task sets и путь submission.
2. [CANN Bench: Benchmarking Agent Generated Kernels against Real NPU and Algorithmic Limits](#) — техническая статья и заявленные design criteria.
3. [Официальная benchmark specification](#) — этапы, score и состав уровней.
4. [Leaderboard](#) — публичная поверхность сравнения результатов.

Опубликованный масштаб и фактический масштаб этого run

Опубликованный benchmark

53 оператора · 1060 открытых case'ов · L1-L4 · FP16/BF16/FP32/INT8. Техническая статья также описывает hidden split для leaderboard.

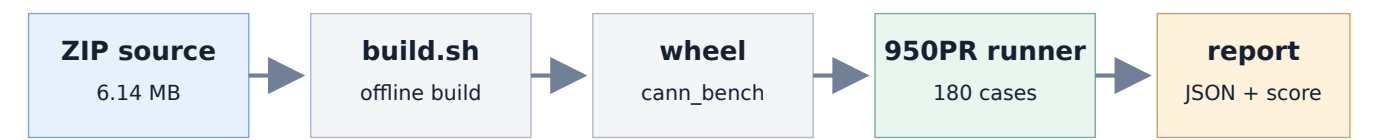
Наш private run

9 операторов · 180 открытых case'ов · весь L1 + один L2 · FP16/BF16/FP32 и mask INT8/UINT8 paths.

Следствие: этот результат подходит для оценки полного L1-покрытия ruasc и экспериментального RMSNorm, но не является «полным CANNBench» по всем 53 операторам и не сопоставим напрямую с leaderboard total без одинакового task set.

3. Что было «скормлено» evaluator’у

В API CANNBench был загружен приватный ZIP `rms_norm_hybrid.zip`. CANNBench валидировал архив, поставил job в очередь, распаковал source directory, выполнил `build.sh`, установил полученный wheel и обнаружил callable’ы из Python package `cann_bench`.



Evaluator получает исполняемый artifact; генератор и его transcript остаются вне этой границы.

Идентичность submission

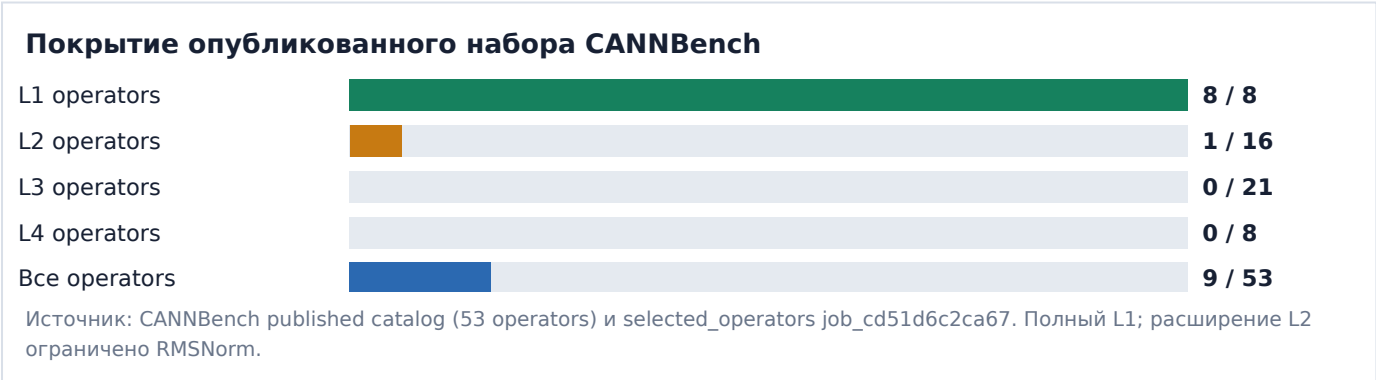
Поле	Значение
Submission ID	sub_d13af9d97a79
Job ID	job_cd51d6c2ca67
Tag	pyasc-v2-full-8l1-rmsnorm-20260901
Visibility	private
Target	950pr
ZIP SHA-256 / site content_hash	39e3d6a4af088a2711e44fc4335241dde0b3884aa6cfe0a0deea99ab7250608e
pyasc source	compiler-team/pyasc , branch <code>v2</code> , commit <code>ac1222a48c8914d3f81297c7570d1a84f0f26778</code>
LLVM reference	LLVM 20 snapshot 86b69c31642e98f8357df62c09d118ad1da4e16a

Содержимое архива

- `build.sh`, `setup.py`
 - `merge_wheels.py`, `verify_wheel.py`
 - 9 operator modules + `__init__.py`
 - `_pyasc_runtime.py`
- vendored pyasc CPython 3.12/x86_64 wheel
 - vendored pybind11 wheel
 - pyasc license и README
 - SHA-256 manifest runtime wheel

Чего в архиве не было: prompt’ов, OpenCode session exports, worker logs, evaluator feedback history и декларации «этот файл сгенерирован GLM-5.2». Поэтому сам submission не переносил полную цепочку происхождения.

4. Охват и список операторов



#	Operator / callable	Level	Семантика	Наблюдавшиеся device-kernel entry names
1	Exp exp	L1	Параметризованная экспонента	_exp_kernel
2	ForeachAddcddivScalar foreach_addcddiv_scalar	L1	List-wise add + div × scalar	_addcddiv_scalar_kernel
3	ForeachNorm foreach_norm	L1	Нормы p для списка tensors	_reduce_*; _finalize_*; _zero_f32_kernel
4	Gelu gelu	L1	GELU, erf/tanh modes	_gelu_erf_kernel_u1; _gelu_tanh_kernel
5	MaskedScale masked_scale	L1	x × mask × scale	_cast_mask_to_f32; _masked_scale_kernel
6	Mish mish	L1	x × tanh(softplus(x))	_mish_kernel
7	Sigmoid sigmoid	L1	Логистическая функция	_sigmoid_kernel
8	SwiGlu swi_glu	L1	SiLU(x ₀) × x ₁ после split	_swiglu_2d_kernel
9	RmsNorm rms_norm	L2	RMS normalization	_rms_norm_kernel; _rms_norm_fallback_kernel

«Ядро» в отчёте используется в двух смыслах: benchmark operator (9 callable’ов) и фактический NPU kernel entry (несколько entry на некоторых операторах). CANNBench выставляет score на уровне operator и case, а profiler фиксирует фактически исполненные device kernels.

4.1 Матрица correctness по 180 case'ам

pass fail

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Exp	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
ForeachAddcdvScalar	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
ForeachNorm	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
Gelu	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
MaskedScale	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
Mish	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
SwiGlu	pass	pass	pass	pass	pass	pass	pass	pass	fail	pass	pass	fail	pass	pass	pass	pass	pass	pass	pass	pass
Sigmoid	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass
RmsNorm	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass	pass

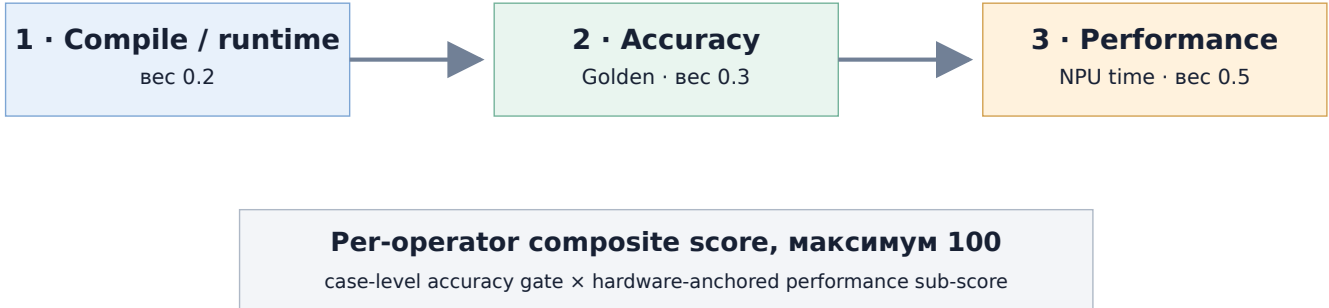
Источник: eval1/report.json, job_cd51d6c2ca67. Красные клетки: SwiGlu case 9 и 12.

SwiGlu case 9. BF16, shape [363, 367, 14], dim=-1. Host-side narrow(...).contiguous() попал в aclnnInplaceCopy и завершился CANN runtime error.

SwiGlu case 12. BF16, shape [1000003, 2], dim=1, special Inf input. Численные ошибки MERE/MARE нулевые, но позиции NaN не совпали с Golden.

5. Что именно измеряет CANNBench

CANNBench оценивает **результат работы программной системы** — реализованный оператор, который можно собрать и запустить на NPU. Модель и skills находятся upstream: они могут создавать или улучшать исходники, но в timed path не участвуют.



$$\text{score}_i = (T_{\text{baseline},i} - T_{\text{HW},i}) / ((T_{\text{candidate},i} - T_{\text{HW},i}) + (T_{\text{baseline},i} - T_{\text{HW},i}))$$

$$\text{OperatorScore} = [0.2 \cdot \delta_{\text{runtime}} + (1/N) \cdot \sum_i \delta_{\text{accuracy},i} \cdot (0.3 + 0.5 \cdot \text{score}_i)] \cdot 100$$

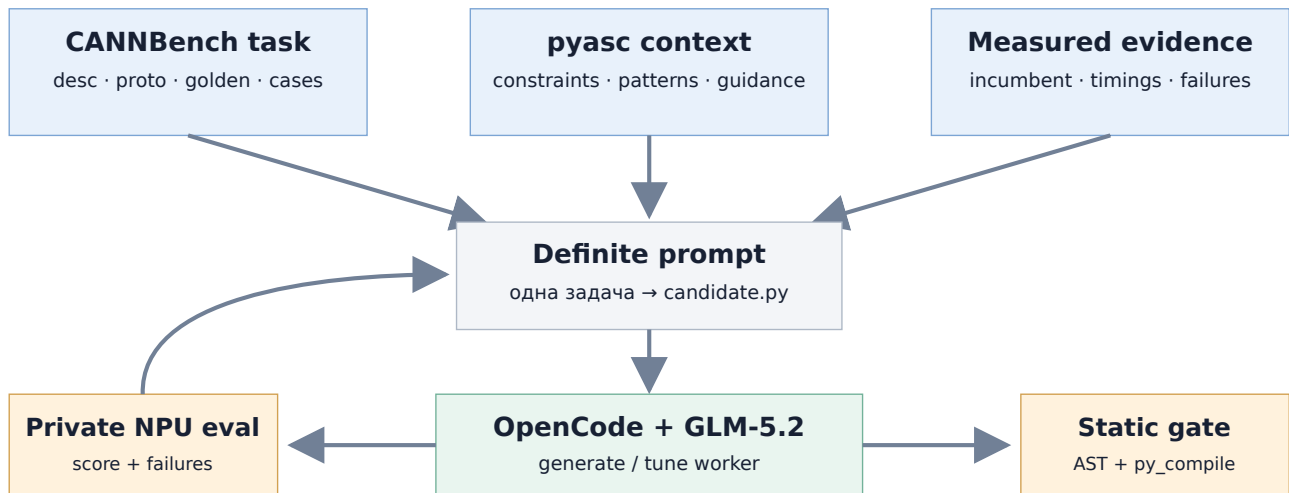
- **T_candidate** берётся из kernel-level profiler на NPU, а не из wall-clock времени Python.
- **T_baseline** — versioned out-of-the-box PyTorch-on-Ascend/CANN reference.
- **T_HW** — аналитический Hardware-Anchored Performance limit для конкретного case.
- Неверный case не получает functional/performance contribution; runtime failure также уменьшает compile/runtime component.

Ответ на вопрос «модель или модель + skills?» CANNBench непосредственно оценивает ни то и ни другое: он оценивает итоговый operator artifact. На leaderboard можно позиционировать связку *base model + agent/skill*, если она честно указана в metadata и подкреплена воспроизводимым generation trail.

6. Как создавались ядра

Для L1 использовался локальный worker-driven цикл в репозитории pyasc-skill-stack. Драйвер формировал определённый self-contained prompt и запускал локальный OpenCode worker с моделью dashscope/glm-5.2.

```
opencode run --pure -m dashscope/glm-5.2 <definite prompt>
```



Цикл: сгенерировать → статически проверить → отправить immutable private submission → вернуть worker'y digest → принять только корректный/лучший candidate.

Что попадало в definite prompt

- desc.md, proto.yaml, golden.py, сводка всех 20 cases;
- ограничения pyasc asc2, UB/alignment notes, известные codegen hazards;
- проверенный reference module Sigmoid для generation-задач;
- для tuning — текущий source, per-case NPU timings, baseline, НАР и предыдущие ошибки;
- требование писать только полный candidate.py с точной сигнатурой.

6.1 Трассировка финальных модулей

Operator	Режим	Трассируемый источник	Итерация	SHA-256 prefix
Exp	GLM tune	20260828_180653/exp-tune	iter2/3	6dbde956aa9f
Sigmoid	GLM tune	20260828_180653/sigmoid-tune	iter2/3	14bcbcb71f4f
Mish	GLM tune	20260828_180653/mish-tune	iter3	edcf4f15b083
Gelu	GLM tune	20260828_195630/gelu-tune	iter3	c139b5db752c
MaskedScale	GLM generate	20260828_234415/masked_scale-generate	iter3	587df4ad6d3b
SwiGlu	GLM generate	20260828_222246/swi_glu-generate	iter2	d4c98c318f02*
ForeachAddcdivScalar	GLM generate	20260828_195630/...-generate	iter1	241aa7fd8a07
ForeachNorm	GLM generate	20260828_222246/...-generate	iter3	58836e1ab059
RmsNorm	code-assisted	20260901_code_rms_norm/candidate_builtin.py	hybrid	cf9080043186

* SwiGlu worker candidate и canonical source логически идентичны; canonical copy содержит дополнительный завершающий newline, поэтому byte-level hash отличается.

Подтверждённый provenance финального 9-ор package

8 L1 · OpenCode / GLM-5.2 traceable

1 L2

88.9% package operators

11.1% code-assisted

Источник: byte-level matching canonical modules к archived candidate.py и worker summaries.

Честная граница утверждения. Нельзя называть весь 9-ор package результатом только GLM-5.2: GLM-run для RMSNorm не создал candidate, а принятый RMSNorm был получен в отдельном code-assisted/hybrid цикле. Для «чистого» сравнения GLM-5.2 + skill-stack следует либо ограничить submission восемью L1, либо заново получить RMSNorm через тот же фиксированный worker protocol.

7. Как система понимает, что ей не «подсунули» готовый ответ

Она не определяет автора. Вместо этого CANNBench проверяет, что вычисление действительно выполнено submitted NPU kernel и что реализация не эксплуатирует evaluator. Это защита от reward hacking, а не детектор AI authorship.

Контроль	Что предотвращает	Тип
Golden accuracy + fresh-input retry	Неверную математику, кэширование заранее вычисленного результата	автоматический
Input address rotation	Кэш по data_ptr() и повторное возвращение старого output	автоматический
TorchOpGuard / DeviceResidencyGuard	Вызов PyTorch/ATen/CANN built-ins вместо собственного kernel, CPU offload	автоматический
NPU profiler attribution	Performance score без фактического исполнения submitted NPU kernel	автоматический
Timing/profiler integrity rules	Monkey-patching synchronize, profiler и timing API	автоматический + review
Contiguous output check	Возврат view/FakeTensor без реального вычисления	автоматический
Запрет host-side layout/data transforms	Перенос значимой части семантики в wrapper	частично ручной review
Hidden cases для leaderboard	Подгонку по опубликованным shapes/values	protocol-level

Официальные запрещённые классы поведения

- SUB-BEH-001: встроенные PyTorch/torch_npu compute API;
 - SUB-BEH-002: существенная обработка I/O tensor в wrapper;
 - SUB-BEH-003: routing к встроенному одноимённому CANN op;
 - SUB-BEH-004: CPU fallback / отсутствие submitted NPU kernel.
- SUB-BEH-005: cached/fixed output, case hardcoding;
 - SUB-BEH-006: подмена profiler/timing/runtime;
 - SUB-BEH-007: FakeTensor или lazy wrapper;
 - обязательный реальный contiguous torch.Tensor output.

Подробная официальная граница: [«算子提交行为与禁止规则»](#) / [Submission behavior and prohibited rules](#).

Наш результат: anti_cheat_failed_cases = 0. В profiler CANNBench видны custom entry names pyasc kernels. Это подтверждает допустимый execution path в данном прогоне, но не заменяет provenance модели.

7.1 Что нужно для доказуемой AI-атрибуции

Официальная specification предусматривает поля `base_model` и `agent_skill`, а также указывает `model information`, `full prompt` и `operator source` как входы официальной оценки. Но это декларативные данные: `benchmark` не может восстановить их из машинного кода.

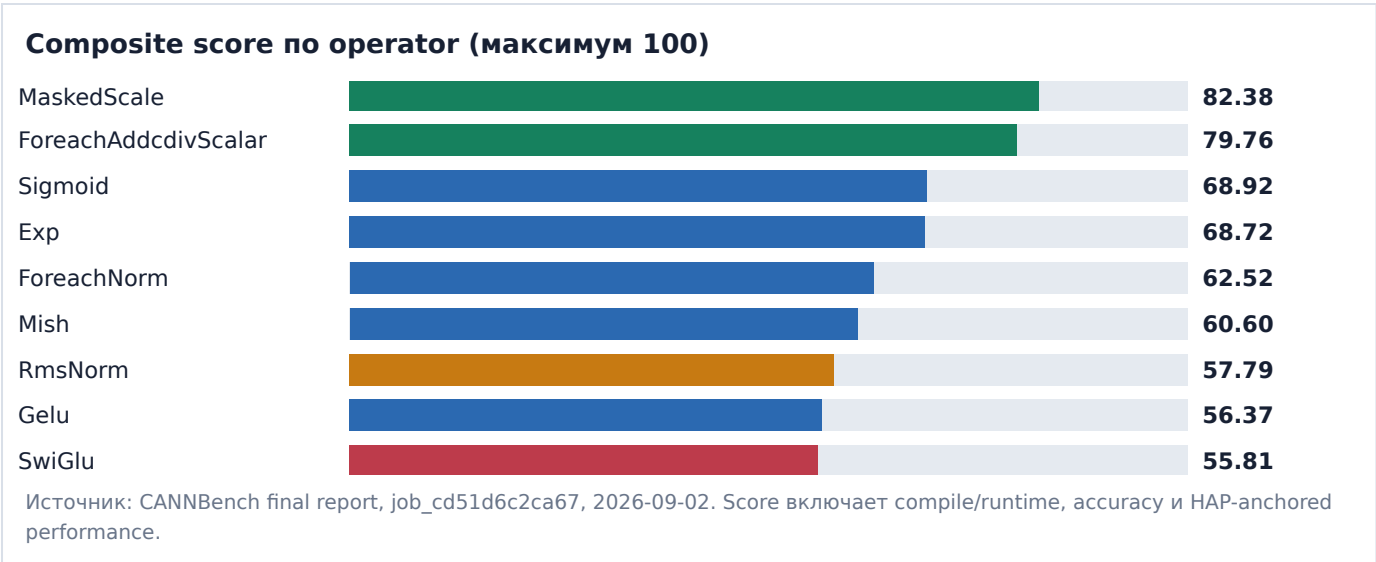
Слой доказательности	В текущем run	Что добавить
Report metadata	<code>base_model = ""</code> <code>agent_skill = ""</code>	<code>dashscope/glm-5.2</code> ; <code>pyasc-skill-stack</code> ; отдельно отметить <code>mixed RMSNorm</code>
Prompt trail	локально сохранён	Включить <code>SHA-256 prompts</code> и <code>immutable archive/session export</code>
Candidate trail	<code>candidate.py + digest</code> сохранены	Manifest: <code>operator</code> → <code>run/iter</code> → <code>candidate hash</code> → <code>acceptance reason</code>
Toolchain identity	<code>pyasc/LLVM pinned</code>	Добавить <code>wheel hash</code> , <code>Python/CANN target</code> и <code>build recipe</code> в <code>manifest</code>
Human intervention	не представлено evaluator’у	Явный <code>log</code> : какие правки сделаны человеком/Codex после <code>worker output</code>
Submission binding	<code>ZIP content_hash</code> есть	Подписать <code>manifest</code> и связать его с <code>CANNBench submission ID/hash</code>

Минимальный provenance manifest. Для каждого `operator`: `task version/hash`, `exact prompt hash`, `model provider/id`, `agent/skill versions`, `command line`, `session ID/export`, `candidate hash`, `evaluator digest`, принятые ручные изменения и `final source hash`. На верхнем уровне — `pyasc commit`, `LLVM commit`, `submission ZIP hash` и `CANNBench job ID`.

Если цель исследования — сравнить *модель*, фиксируются `prompt`, `skills`, `tool budget` и число итераций. Если цель — сравнить *agentic system*, модель считается компонентом системы, а `score` атрибутируется всей связке: `модель + skills + orchestration + compiler/runtime + feedback loop`.

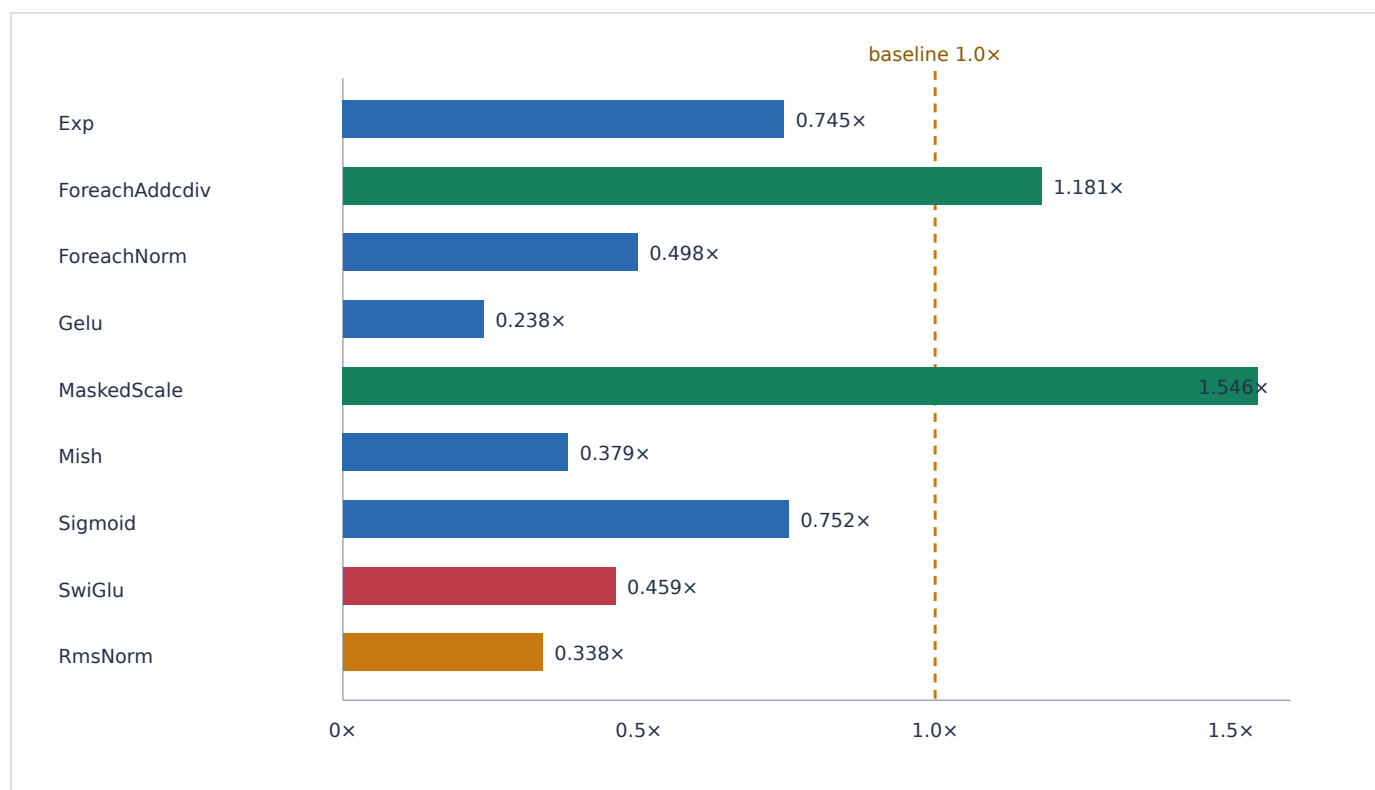
8. Результаты аппаратного прогона

Operator	Cases	Score /100	Avg speedup	Compile	Accuracy	Perf
Exp	20/20	68.72	0.745×	20	30	18.72
ForeachAddcdivScalar	20/20	79.76	1.181×	20	30	29.76
ForeachNorm	20/20	62.52	0.498×	20	30	12.52
Gelu	20/20	56.37	0.238×	20	30	6.37
MaskedScale	20/20	82.38	1.546×	20	30	32.38
Mish	20/20	60.60	0.379×	20	30	10.60
Sigmoid	20/20	68.92	0.752×	20	30	18.92
SwiGlu	18/20	55.81	0.459×	19	27	9.81
RmsNorm	20/20	57.79	0.338×	20	30	7.79
Итого / aggregate	178/180	592.87	0.577× geo	—	—	—



8.1 Производительность относительно baseline

Средний speedup выше 1× означает преимущество над baseline; ниже 1× — замедление. Поскольку composite score включает 50 points за compile+accuracy, корректный, но медленный оператор может иметь score выше 50.



Avg speedup по успешным case'ам. Aggregate geometric mean по run: 0.5772x. Источник: eval1/report.json.

Интерпретация

- **MaskedScale** — сильнейший результат: 20/20, score 82.38, 1.546x.
- **ForeachAddcdivScalar** — 20/20, score 79.76, 1.181x.
- **Exp** и **Sigmoid** близки друг к другу и сохраняют полную correctness, но уступают baseline примерно на четверть.
- **Gelu**, **Mish**, **RmsNorm** функционально полны, но требуют системного perf tuning; особенно дорогие fallback paths RMSNorm.
- **SwiGlu** сначала требует закрытия двух correctness gaps; perf tuning до этого вторичен.

8.2 Среда и воспроизводимость

Компонент	Зафиксированное значение
CANNBench framework/tasks	1.0.0 / 1.0.0
Runner CANNBench ref	e8609f6b5891361fb6f0afb141eb8909c1b4a59b
Hardware	Ascend950PR_957c × 1
CANN / driver	9.1.0 / cann-9.1.0
Python	3.12.13, x86_64
PyTorch / torch_npu	2.10.0+cpu / 2.10.0.post4
pyasc	branch v2, commit ac1222a48c8914d3f81297c7570d1a84f0f26778
pyasc runtime wheel hash	27e675fe2446b9ff8b2841688218c2d6b3ace877a3c4be491d0cba68af4dfcdd
LLVM	20 snapshot 86b69c31642e98f8357df62c09d118ad1da4e16a
Eval code / timestamp	cann_final_eval_20260902_032606 · 2026-09-02T03:26:06Z

QEMU использовался только для build portability. На локальной ARM-машине x86_64 CPython 3.12 pyasc runtime wheel собирался через Docker Buildx/QEMU с pinned LLVM. Измерения производительности выполнялись не в QEMU и не в CaModel, а реальным CANNBench runner на Ascend 950PR.

CaModel role: быстрая локальная проверка codegen/UB/компилируемости и отбраковка кандидатов. **CANNBench role:** authoritative correctness и kernel timing на настоящем NPU. Поэтому CaModel полезен как inner loop, но не заменяет score CANNBench.

9. Ограничения и корректная интерпретация

1. **Private run, не leaderboard entry.** Submission имел `is_private=true`; место в публичном рейтинге не создавалось.
2. **Это не все 53 оператора.** Охвачен полный L1 и один L2: 9/53 operators.
3. **Открытые 20-case наборы.** В job фактически оценено 20 cases/operator. Hidden leaderboard split, описанный в статье, этим report не подтверждён.
4. **Пустые model/skill metadata.** Финальный CANNBench JSON не атрибутирует результат GLM-5.2 или pyasc-skill-stack.
5. **Mixed provenance.** Восемь L1 трассируются к OpenCode/GLM-5.2; RMSNorm — отдельный code-assisted candidate.
6. **Сравнивать только одинаковые task/hardware versions.** Overall 592.87 — сумма по выбранным 9 operators; она не равна leaderboard total для полного набора.
7. **Ноль anti-cheat failures не доказывает authorship.** Он показывает отсутствие обнаруженных forbidden execution paths.
8. **Terminal status корректен.** `correctness_failed` отражает 2 failed cases и не отменяет сохранённые scores остальных operators.

Неудачный follow-up SwiGlu

Отдельный job `job_0f5fdee4443a` проверял kernel-only fallback для SwiGlu. Он завершился 0/20 с одинаковым subprocess `rc=1`; site report не содержал stderr/traceback. Локальный pyasc/CaModel compile проходил. Кандидат отклонён, canonical SwiGlu восстановлен к подтверждённой версии 18/20.

Этот 0/20 результат нельзя использовать как измерение качества алгоритма нового fallback: evaluator не дошёл до case-level accuracy diagnostics. Он пригоден только как evidence неуспешного запуска конкретного submission.

10. Рекомендации для следующего submission

Приоритет А — сделать результат научно атрибутируемым

1. Выбрать объект сравнения: **чистый GLM-5.2 + pyasc-skill-stack** или **смешанная agentic system**.
2. Для чистого варианта заново провести RMSNorm тем же OpenCode worker protocol либо исключить RMSNorm и подать полный 8-op L1.
3. Заполнить CANNBench metadata: base_model, agent_skill, harness/orchestrator version.
4. Сгенерировать immutable provenance manifest и связать его hash с submission tag.

Приоритет В — закрыть correctness

1. Сначала контрольный one-op run прежнего SwiGlu 18/20 для проверки runner/packaging.
2. Заменить host-side Slice/contiguous path case 9 на NPU-kernel data movement, не нарушающий SUB-BEH-002.
3. Для case 12 явно проверить IEEE Inf/NaN propagation BF16 против Golden.
4. Только после 20/20 запускать perf tuning SwiGlu.

Приоритет С — performance loop

1. Профилировать Gelu, Mish и RMSNorm по case classes; отделить launch-bound, DMA-bound и UB/codegen-limited группы.
2. Использовать CaModel/LLVM для inner-loop compile/UB проверки, но принимать perf изменения только по NPU run.
3. Сохранять одинаковый task version, hardware class и CANN version для всех сравнений.
4. После полного 20/20 выполнить один 9-op validation run и затем отдельный public/leaderboard submission.

Рекомендуемый следующий benchmark artifact: 8-op L1, все prompt/session/candidate hashes, metadata GLM-5.2 + pyasc-skill-stack, 160/160 correctness, затем hidden/leaderboard evaluation. RMSNorm можно публиковать отдельным mixed-system extension до его чистой регенерации.

Приложение А. Локальные evidence paths

Назначение	Путь относительно workspace
Финальный CANNBench report	integrations/cannbench/workers/runs/20260901_full_official/eval1/report.json
Job API payload	.../eval1/site_job.json
Submission API payload	.../eval1/site_submission.json
Stage logs	.../eval1/site_logs.json
Compact result	.../eval1/summary.json
Uploaded ZIP	integrations/cannbench/.uploads/rms_norm_hybrid.zip
Generation/tuning trails	integrations/cannbench/workers/runs/20260828_*/
RMSNorm hybrid candidate	integrations/cannbench/workers/runs/20260901_code_rms_norm/candidate_builtin.py
SwiGlu failed follow-up	.../eval2_swi_glu_fix/
Canonical submission sources	integrations/cannbench/submission/

Ключевые идентификаторы

Объект	SHA-256 / ID
Uploaded ZIP	39e3d6a4af088a2711e44fc4335241dde0b3884aa6cfe0a0deea99ab7250608e
Vendored pyasc runtime wheel	27e675fe2446b9ff8b2841688218c2d6b3ace877a3c4be491d0cba68af4dfcdd
pyasc source commit	ac1222a48c8914d3f81297c7570d1a84f0f26778
LLVM snapshot	86b69c31642e98f8357df62c09d118ad1da4e16a
Submission / Job	sub_d13af9d97a79 / job_cd51d6c2ca67

Приложение В. Источники

1. [CANNBench official portal](#). Позиционирование, task sets и схема запуска.
2. [CANNBench leaderboard](#). Публичная поверхность сравнения submissions.
3. [CANN/cann-bench official repository](#). Source of truth для harness, tasks и документации.
4. [Benchmark specification](#). Три измерения, HAP score, levels и report metadata.
5. [Submission behavior and prohibited rules](#). SUB-BEH-001...007 и anti-cheat boundary.
6. [Submission input/interface specification](#). build.sh, wheel, callable discovery и runtime constraints.
7. [Auto-pipeline agent integration guide](#). Разделение Prompt → Generator → Artifact → Converter → Submission.
8. Gao et al., “CANN Bench: Benchmarking Agent Generated Kernels against Real NPU and Algorithmic Limits”, [arXiv:2607.20518](#).
9. [compiler-team/pyasc, branch v2](#). Runtime/compiler source evaluated in this work.

Методика подготовки отчёта

Официальные statements сверены с CANNBench portal, paper и repository documentation. Числа и environment fields извлечены из сохранённого site report без пересчёта. Provenance определён сравнением SHA-256 canonical sources с archived worker candidates и чтением worker summaries/logs. Все выводы, выходящие за прямые поля источников, помечены как интерпретация или рекомендация.

Отчёт подготовлен 2 сентября 2026 года в workspace `pyasc-skill-stack`. HTML является источником печатного PDF. Ни plan-файл, ни git history не изменялись; commit не создавался.